Universidad del Valle de Guatemala

Facultad de Ciencias y Humanidades

Programación Orientada a Objetos – Sección 21

Docente – Kevin Antonio Velásquez Aguilar



Ejercicio 4. Modelación con herencia

Cristhopper Alejandro López Paz - 26544

Ingeniería en Ciencia de la Computación y Tecnologías de la Información

28/09/2026

Análisis:

1. **Requisitos Funcionales**

- El sistema guardará el inventario de la flota con los datos comunes (placa, marca, modelo, tarifa diaria y disponibilidad). También almacenará las características propias de cada categoría y mantendrá un acumulador del dinero total generado.
- Permitirá registrar vehículos, cotizar alquileres por placa y días, confirmar rentas y registrar devoluciones. Al final del día, generará un reporte de la flota y los ingresos acumulados.
- No se aceptarán placas repetidas o vacías, y todo valor numérico debe ser positivo. Las cotizaciones no alterarán ingresos ni estados de los vehículos. No se puede alquilar un vehículo ocupado ni devolver uno disponible, y los errores de entrada no deben cerrar el programa de forma abrupta.

2. Clases

Superclase: Vehiculo Decidí crear una superclase abstracta para centralizar los datos compartidos y evitar repetir código. Tiene como atributos protegidos la placa (String), marca (String), modelo (int), tarifaDiaria (double) y el estado disponible (boolean). Su constructor recibe los datos básicos y define la disponibilidad en true por defecto. Además, define el método público abstracto cotizarAlquiler(int dias), obligando a cada subclase a programar su propia fórmula matemática de cobro.

Subclase: Automovil Hereda de Vehiculo. Agregué los atributos privados pasajeros (int) y esAutomatico (boolean) porque es lo que buscan las familias. Sobrescribe el método cotizarAlquiler(dias) calculando la tarifa base y sumando un recargo de Q50 por cada día de alquiler si la transmisión es automática.

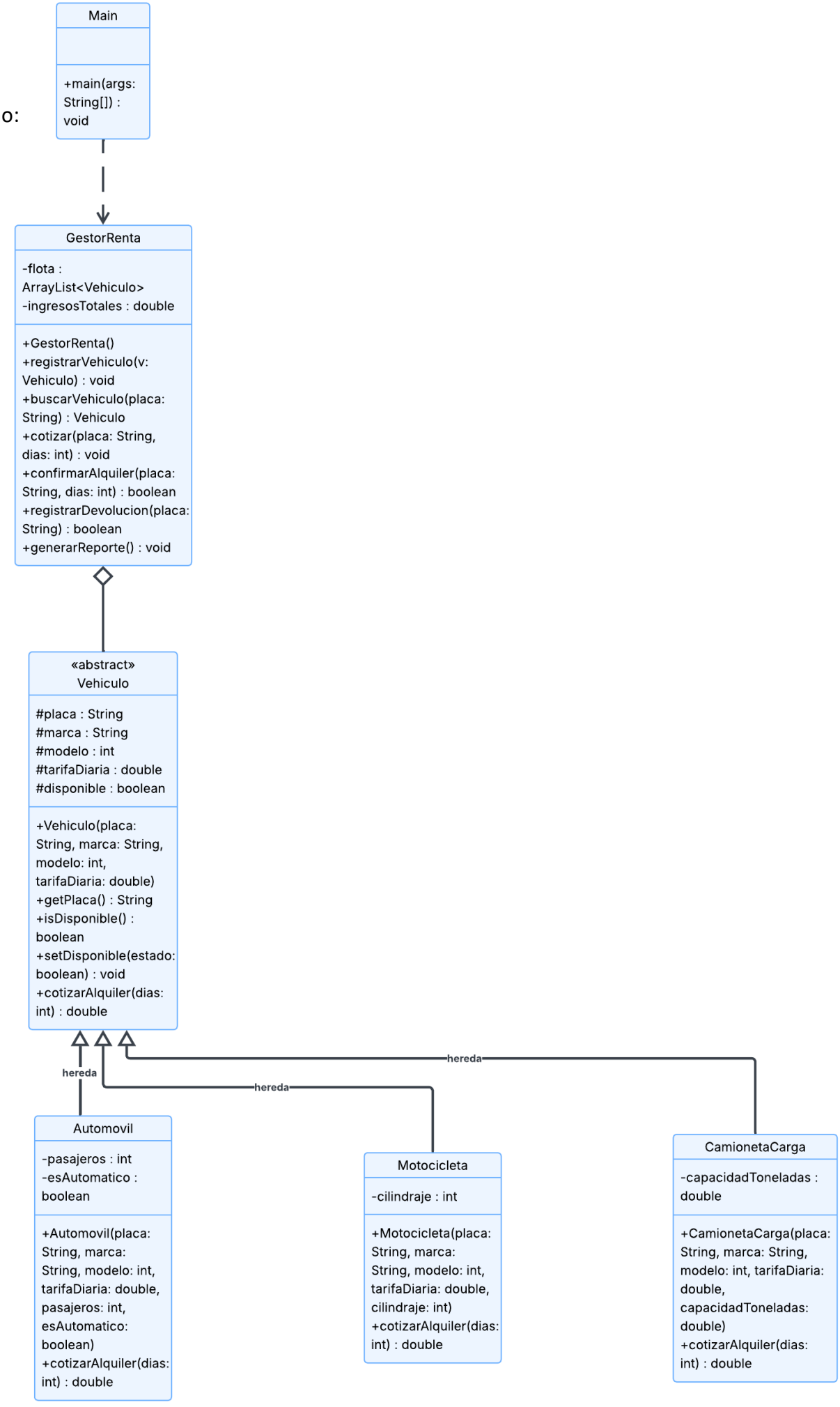
Subclase: Motocicleta También hereda de Vehiculo y añade el atributo privado cilindraje (int). Al sobrescribir cotizarAlquiler(dias), calcula el costo base y evalúa si el motor supera los 250 cc. Si es así, suma un recargo fijo de Q75 una sola vez, sin importar la duración del alquiler.

Subclase: CamionetaCarga Hereda de Vehiculo y su atributo privado es capacidadToneladas (double, para aceptar decimales). En su método cotizarAlquiler(dias), calcula el costo base más un recargo por peso. Este recargo suma Q100 por cada tonelada de capacidad, multiplicado por los días de alquiler solicitados.

Clase Administradora: GestorRenta Esta clase centraliza la lógica del negocio para proteger los datos financieros. Sus atributos privados son una lista para la flota y un acumulador double para ingresosTotales. Al usar la opción "Cotizar", busca el vehículo y llama a su método de cobro, mostrando el total en pantalla sin afectar el dinero ni el estado de la unidad. En "Confirmar Alquiler", verifica que el vehículo esté disponible, lo marca como ocupado false y suma el dinero al acumulador. Las devoluciones simplemente regresan el estado a disponible true sin involucrar cálculos.

3. Justificación del Diseño: Esta organización aprovecha la herencia y el polimorfismo. Al delegar las particularidades a las subclases, la clase administradora no necesita saber qué tipo de vehículo está cobrando, solo busca la placa y le pide al objeto que calcule su tarifa. Si en el futuro RentaMovil incorpora una nueva categoría, el sistema base no sufrirá alteraciones graves. Solo bastará crear una nueva subclase que herede de Vehiculo, asignar sus atributos y programar su método de cobro, manteniendo el resto de las operaciones intactas.

Diseño:


Main
+main(args: String[]) : void
GestorRenta
-flota : ArrayList<Vehiculo>
-ingresosTotales : double
+GestorRenta()
+registrarVehiculo(v: Vehiculo) : void
+buscarVehiculo(placa: String) : Vehiculo
+cotizar(placa: String, dias: int) : void
+confirmarAlquiler(placa: String, dias: int) : boolean
+registrarDevolucion(placa: String) : boolean
+generarReporte() : void
«abstract»
Vehiculo
#placa : String
#marca : String
#modelo : int
#tarifaDiaria : double
#disponible : boolean
+Vehiculo(placa: String, marca: String, modelo: int, tarifaDiaria: double)
+getPlaca() : String
+isDisponible() : boolean
+setDisponible(estado: boolean) : void
+cotizarAlquiler(dias: int) : double
hereda
hereda
hereda
Automovil
-pasajeros : int
-esAutomatico : boolean
+Automovil(placa: String, marca: String, modelo: int, tarifaDiaria: double, pasajeros: int, esAutomatico: boolean)
+cotizarAlquiler(dias: int) : double
Motocicleta
-cilindraje : int
+Motocicleta(placa: String, marca: String, modelo: int, tarifaDiaria: double, cilindraje: int)
+cotizarAlquiler(dias: int) : double
CamionetaCarga
-capacidadToneladas : double
+CamionetaCarga(placa: String, marca: String, modelo: int, tarifaDiaria: double, capacidadToneladas: double)
+cotizarAlquiler(dias: int) : double

Pruebas:

```
Seleccione una opcion: 3
Ingrese placa: 45
Dias de alquiler: 3
Alquiler confirmado. Total cobrado: Q600.00
Rentamovil
1. Registrar Vehiculo
2. Cotizar Alquiler
3. Confirmar Alquiler
4. Registrar Devolucion
5. Generar Reporte de Flota
6. Salir
Seleccione una opcion:
```

```
Seleccione una opcion: 5
Reporte de Jornada
Automoviles     -> Disponibles: 2 | Ocupados: 0
Motocicletas    -> Disponibles: 2 | Ocupados: 1
Camionetas      -> Disponibles: 2 | Ocupados: 0
Ingresos Totales Acumulados: Q1200.00
-------------------------
```

```
1
Ingrese placa: 45
Ingrese marca: toyota
Ingrese modelo (Anio): 2020
Ingrese tarifa diaria (Q): 300
Cantidad de pasajeros: 2
Es automatico? (1: Si, 2: No): 1
Error: Ya existe un vehiculo con la placa 45
```